



Programmieren

Falk Morgenstern

Leon Johann Brettin

Foliensatz basierend auf Unterlagen von Prof. Huhn, Prof. Meyer und weiteren.
Nur zur Verwendung in der Vorlesung „Programmieren“ an der Ostfalia.
Keine Weitergabe.



Programmieren

3. Enumerations



Einleitung

- Oft möchte man, dass eine Variable nur einige vordefinierte Werte annehmen kann.
- **Farben**: Eine Variable „color“ soll die Farben rot, grün, blau repräsentieren
- **Himmelsrichtungen**: Eine Variable „direction“ soll die Himmelsrichtungen Nord, Süd, West, Ost darstellen.
- Erlaubte Werte kann man als Konstanten deklarieren

```
static final int RED = 0;  
static final int BLUE = 1;  
static final int GREEN = 2;
```

```
static final int NORTH = 0;  
static final int SOUTH = 1;  
static final int WEST = 2;  
static final int EAST = 3;
```





Nachteil der Lösung mit Konstanten

- Compiler kann nicht garantieren, dass einer Variablen nur Werte für Farben oder Himmelsrichtungen zugewiesen werden.
- Die folgenden Zuweisungen entsprechen nicht der Intention, sind jedoch korrekt.

```
int color = NORTH;  
int direction = BLUE;  
color = 25;
```





Enumerationstypen

- In Java 5 wurden deshalb Enumerationstypen eingeführt.
- Ein Enumerationstyp definiert die Menge seiner Werte durch explizite Aufzählung.
- Farben und Himmelsrichtungen können so als **Typen** definiert werden.
- Damit kann man Variablen deklarieren, die nur die Werte von Farben oder Himmelsrichtungen annehmen können.
- Siehe die Klasse `javafx.scene.layout.Priority`, `javafx.scene.input.KeyCode` als Beispiele aus JavaFX.



Deklaration von Enumerationstypen

- Eine Deklaration eines Enumerationstyps besteht aus
 - Schlüsselwort `enum`
 - Typnamen
 - Liste der Werte in geschweiften Klammern

```
enum Color {RED, BLUE, GREEN};  
enum Direction {NORTH, SOUTH, WEST, EAST};
```





Nutzung von Enumerationstypen

- Werte eines Enumerationstyps müssen mit dem Typnamen qualifiziert werden (wie bei Attributen, Methoden von Klassen).
- Compiler kann sicherstellen, dass Variablen eines Enumerationstypen nur Werte dieses Typs zugewiesen werden.

```
Color c = Color.RED;           // erlaubt
Direction d = Direction.NORTH; // erlaubt
c = Direction.NORTH;          // falscher Typ
c = 0;                         // falscher Typ
```





Bemerkungen zu Enumerationstypen

- Wo darf man Enumerationstyp deklarieren?
 - Als eigene Klasse
 - innerhalb einer Klasse auf dem Toplevel, also wie die Deklaration von Klassenvariablen
- Wie verwendet man einen Enumerationstyp in einer anderen Klasse?
 - Werte müssen mit dem Typnamen qualifiziert werden.



Vergleiche

- Aufzählungstypen werden intern als Unterklassen der Spezialklasse Enum realisiert.
- **Test auf Gleichheit** mit `==`, `!=` oder `equals`
- Compiler ordnet jedem Wert eines Enumerationstypen eine Ordinalzahl zu. Die Ordinalzahl gibt die Position des Elements in der Folge wieder, gezählt wird: 0, 1, 2, ...
- **Vergleich** von zwei Variablen ist mit `compareTo` möglich. Verglichen wird die Position in der Definition des Enumerationstyps.
 - `x.compareTo(y)` ist kleiner 0, wenn Wert von x vor Wert von y in Typdefinition steht, größer 0, wenn x nach y steht und gleich 0, wenn x gleich y ist.



Vergleiche: Beispiel

```
Color c = Color.BLUE;
if (c == Color.BLUE) { /* liefert true */ }
if (c != Color.RED) { /* liefert true */ }
if (c.compareTo(Color.GREEN) > 0) { /* liefert false */}
if (c.compareTo(Color.BLUE) >= 0) { /* liefert true */ }
```





Verwendung in switch-Statements

- Enumerationstypen können als case-Marken in einem `switch`-Statement verwendet werden.
- Werte werden dann ohne den qualifizierenden Typnamen geschrieben. Also `RED` statt `Color.RED`.
- Typ wird aus dem Typ des Switch-Ausdrucks abgeleitet.

```
switch (c) {  
    case RED:  
        System.out.println("Die Farbe ist rot."); break;  
    case BLUE:  
        System.out.println("Die Farbe ist blau."); break;  
    case GREEN:  
        System.out.println("Die Farbe ist gruen."); break;  
}
```





Klasse Enum

- Aufzählungstypen werden intern als Unterklassen der Spezialklasse Enum realisiert.
- Die Ordnungszahl gibt die Position des Elements in der Folge wieder. Werte sind 0, 1, 2, ...
- Intern werden Werte eines Enumerationstyps mit dem Konstruktor der Klasse Enum erzeugt, Enum(String name, int ordinal)
- Ein Element eines Aufzählungstyps ist also ein Objekt mit 2 Attributen:
 - **Name**: Ein Bezeichner vom Typ String
 - **Ordinalzahl**: Einer Ordnungszahl vom Typ int
- Jeder **Wert** wird durch ein **Objekt** dargestellt, das nicht kopiert werden kann. (Deshalb Vergleich mit == möglich.)



Klasse Enum

- Von der Klasse Enum werden einige Methoden der geerbt, die wir für Objekte von Enumerationstypen nutzen können.
- Sei die Variable `c` vom Typ `Color`
- `int c.ordinal()`
liefert die Ordinalzahl des Wertes von `c`
- `String c.toString()`
liefert den Namen des Wertes von `c`
- `Color Color.valueOf("RED")`
Wert vom Typ `Color` der zu dem `String` gehört (also z.B. `Color.RED`).
- `Color[] Color.values()`
Array mit den Werten des Enumerationstyps in gegebenen Reihenfolge.



Verwendung in einer for-Schleife

- Das Array, das man als Ausgabe der Methode `values()` erhält, kann man in for-Schleifen nutzen

```
for (Color col : Color.values()) {  
    System.out.println(col.toString() + " = " + col.ordinal());  
}  
// Ausgabe...  
// RED = 0  
// BLUE = 1  
// GREEN = 2
```





Enumerationstypen als Klassen

- Enumerationstypen werden intern als Unterklassen von Enum abgebildet.
- Enumerationstypen sind also Klassen und können neben den Werten auch selbstdefinierte **Attribute**, **Methoden** und **Konstruktoren** aufweisen.



Beispiel: Römische Zahlen

```
enum Roman {  
    I(1), V(5), X(10), L(50), C(100), D(500), M(1000);  
  
    private int value; // Attribute value  
  
    Roman(int val) { // eigener Konstruktor  
        value = val;  
    }  
  
    int getValue() { // Methode  
        return value;  
    }  
}
```





Beispiel: Römische Zahlen

- Wie ist dieser Typ zu verstehen?
 - Deklaration eines Werts `V(5)`
 - Erzeugen eines Objekts `Roman.V` vom Typ `Roman` mit Konstruktor
 - Dabei wird das Attribut `value` auf 5 gesetzt.
 - Das Attribut `value` können wir mit der Methode `getValue()` ausgeben.

```
Roman r = Roman.V;  
System.out.println(r.getValue()); // Ausgabe 5  
System.out.println(r.ordinal()); // Ausgabe 1
```

